Министерство науки и высшего образования Российской Федерации
Федеральное государственное автономное образовательное учреждение
высшего образования
«СЕВЕРО-КАВКАЗСКИЙ ФЕДЕРАЛЬНЫЙ УНИВЕРСИТЕТ»

Comentado [PM1]: https://github.com/Pascoalpm/lab-jupyter-autovalores

Институт Перспективной инженерии
Департамент цифровых и робототехнических систем и электроники

ОТЧЕТ
ПО ПРАКТИЧЕСКОЙ РАБОТЕ № 1
дисциплины «Искусственный Интеллект и Машинное Обучение»

Выполнил: Мендеш Пашкоал Педру
2 курс, группа ИВТ-б-о-24-1,
09.03.01 «Информатика и Вычислительная техника», направленность (профиль) «Программное обеспечение средств вычислительной техники и автоматизированных систем», очная форма обучения

_____
(подпись)

Руководитель практики:
Воронкин Роман А. доцент факультета цифровых, робототехнических систем и электроники института перспективной инженерии.

_____
(подпись)

Отчет защищен с оценкой _____ Дата защиты_____

Ставрополь, 2026 г.

Тема: Работа с Jupyter Notebook, JupyterLab и Google Colab

Цель работы: исследовать базовые возможности интерактивных оболочек Jupyter Notebook, JupyterLab и Google Colab для языка программирования Python.

Этапы выполнения лабораторных работ

1. Изучение теоретического материала по JupyterLab и Google Colab.
2. Создание репозитория на GitHub: https://github.com/Pascoalpm/lab-jupyter-autovalores
3. Клонирование репозитория:
4. Настройка .gitignore для Python, Jupyter.
5. Запуск JupyterLab:
6. Создание ноутбука variante17_autovalores.ipynb.
7. Выполнение заданий

Работа с ячейками и Markdown

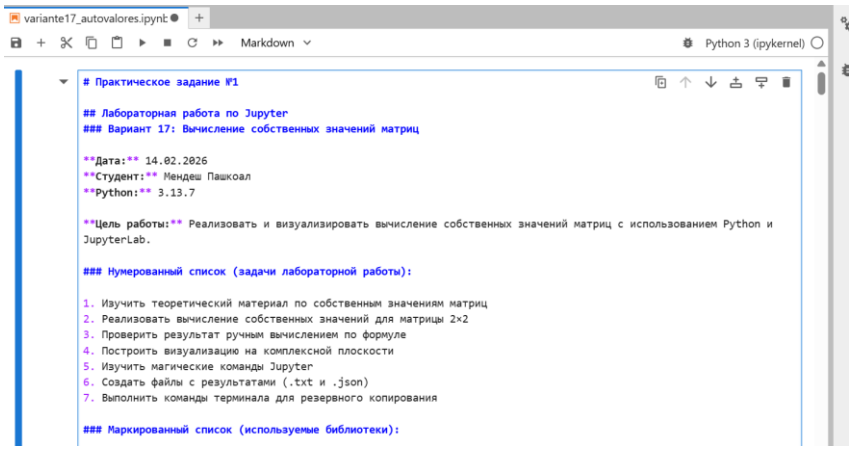

Рисунок 1: Оформить заголовок лабораторной работы

```
### Маркированный список (используемые библиотеки):

- NumPy — для вычисления собственных значений
- Matplotlib — для визуализации графиков
- OS — для работы с файловой системой
- JSON — для сохранения структурированных данных
- Time — для измерения времени выполнения

### Изображение (формула для варианта 17):

Собственные значения матрицы находятся из характеристического уравнения:

![Формула собственных значений](https://github.com/Pascoalpm/lab-jupyter-autovalores/blob/main/Lab%201/Imagem%20do%20exemplo/formula_variante17.png?raw=true)

*Рисунок 1: Характеристическое уравнение det(A − λI) = 0*
```

Рисунок 2: Оформить заголовок лабораторной работы

Markdown Python 3 (ipykernel)

Практическое задание №1

Лабораторная работа по Jupyter

Вариант 17: Вычисление собственных значений матриц

Дата: 14.02.2026
Студент: Мендеш Пашкоал
Python: 3.13.7

Цель работы: Реализовать и визуализировать вычисление собственных значений матриц с использованием Python и JupyterLab.

Нумерованный список (задачи лабораторной работы):

1. Изучить теоретический материал по собственным значениям матриц
2. Реализовать вычисление собственных значений для матрицы 2×2
3. Проверить результат ручным вычислением по формуле
4. Построить визуализацию на комплексной плоскости
5. Изучить магические команды Jupyter

Рисунок 3: Результат выполнения Текта

- Matplotlib — для визуализации графиков
- OS — для работы с файловой системой
- JSON — для сохранения структурированных данных
- Time — для измерения времени выполнения

Изображение (формула для варианта 17):

Собственные значения матрицы находятся из характеристического уравнения:

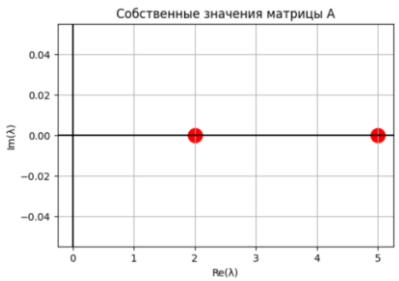

Рисунок 4: Результат выполнения Текта

```
[2]: # Запрос имени пользователя
name = input("Введите ваше имя: ")

# Вывод приветствия
print(f"Привет, {name}! Добро пожаловать в JupyterLab!")

Введите ваше имя:  Пашкоал
Привет, Пашкоал! Добро пожаловать в JupyterLab!
```

Рисунок 5: Запрос имени пользователя

Проверка окружения и установка библиотек

```
variante17_autovalores.ipynb
Code                                Python 3 (ipykernel)
[ ]: # Проверка версии Python и установка необходимых библиотек
import sys
print(f"Версия Python: {sys.version}")

# Установка библиотек (если не установлены)
%pip install numpy matplotlib
```

Рисунок 6: Проверить версию Python и установить необходимые библиотеки

```
Версия Python: 3.13.7 (tags/v3.13.7:bcee1c3, Aug 14 2025, 14:15:11) [MSC v.1944 64 bit (AMD64)]
Requirement already satisfied: numpy in c:\users\hp\appdata\local\programs\python\python313\lib\site-packages (2.4.2)
Requirement already satisfied: matplotlib in c:\users\hp\appdata\local\programs\python\python313\lib\site-packages (3.10.8)
Requirement already satisfied: contourpy>=1.0.1 in c:\users\hp\appdata\local\programs\python\python313\lib\site-packages (from matplotl
ib) (1.3.3)
Requirement already satisfied: cycler>=0.10 in c:\users\hp\appdata\local\programs\python\python313\lib\site-packages (from matplotlib)
(0.12.1)
Requirement already satisfied: fonttools>=4.22.0 in c:\users\hp\appdata\local\programs\python\python313\lib\site-packages (from matplot
lib) (4.61.1)
Requirement already satisfied: kiwisolver>=1.3.1 in c:\users\hp\appdata\local\programs\python\python313\lib\site-packages (from matplot
lib) (1.4.9)
Requirement already satisfied: packaging>=20.0 in c:\users\hp\appdata\local\programs\python\python313\lib\site-packages (from matplotli
b) (26.0)
Requirement already satisfied: pillow>=8 in c:\users\hp\appdata\local\programs\python\python313\lib\site-packages (from matplotlib) (1
2.1.1)
Requirement already satisfied: pyparsing>=3 in c:\users\hp\appdata\local\programs\python\python313\lib\site-packages (from matplotlib)
(3.3.2)
Requirement already satisfied: python-dateutil>=2.7 in c:\users\hp\appdata\local\programs\python\python313\lib\site-packages (from matp
lotlib) (2.9.0.post0)
Requirement already satisfied: six>=1.5 in c:\users\hp\appdata\local\programs\python\python313\lib\site-packages (from python-dateutil>
=2.7->matplotlib) (1.17.0)
Note: you may need to restart the kernel to use updated packages.
```

Рисунок 7: Результат проверки

Индивидуальное задание

Задание 1

17. Формула для вычисления собственных значений матрицы:

```
Markdown

### Теоретическая справка

Собственные значения матрицы **A** находятся из характеристического уравнения:

$$det(A - \lambda I) = 0$$

где:
- **A** - исходная матрица
- **λ** - собственное значение (лямбда)
- **I** - единичная матрица
- **det** - определитель

Для матрицы 2×2:
$$A = \begin{pmatrix} a & b \\ c & d \end{pmatrix}$$

Характеристическое уравнение:
$$\lambda^2 - (a+d)\lambda + (ad - bc) = 0$$

Решение:
$$\lambda_{1,2} = \frac{(a+d) \pm \sqrt{(a+d)^2 - 4(ad-bc)}}{2}$$
```

Рисунок 8: формула для матрицы 2×2.

Code

Теоретическая справка

Собственные значения матрицы **A** находятся из характеристического уравнения:

$$det(A - \lambda I) = 0$$

где:

- **A** - исходная матрица
- **λ** - собственное значение (лямбда)
- **I** - единичная матрица
- **det** - определитель

Для матрицы 2×2:

$$A = \begin{pmatrix} a & b \\ c & d \end{pmatrix}$$

Характеристическое уравнение:

$$\lambda^2 - (a + d)\lambda + (ad - bc) = 0$$

Решение:

$$\lambda_{1,2} = \frac{(a + d) \pm \sqrt{(a + d)^2 - 4(ad - bc)}}{2}$$

Рисунок 9: Результат выполнения кода

Построить визуализацию собственных значений для различных типов матриц на комплексной плоскости.

```
import numpy as np
import matplotlib.pyplot as plt
%matplotlib inline

print("ВЫЧИСЛЕНИЕ СОБСТВЕННЫХ ЗНАЧЕНИЙ МАТРИЦЫ")
print("="*60)

# Матрица 2x2 (exemplo)
A = np.array([[4, 2],
              [1, 3]])

print("Матрица A:")
print(A)

# Вычисление собственных значений
eigenvalues = np.linalg.eigvals(A)
print(f"\nСобственные значения: {eigenvalues}")

# Ручное вычисление
a, b, c, d = A.flatten()
trace = a + d
det = a*d - b*c

lambda1 = (trace + np.sqrt(trace**2 - 4*det)) / 2
lambda2 = (trace - np.sqrt(trace**2 - 4*det)) / 2

print(f"\nРучное вычисление:")
print(f"λ₁ = {lambda1:.6f}")
print(f"λ₂ = {lambda2:.6f}")
print(f"Результаты совпадают: {np.isclose(lambda1, eigenvalues[0]) and np.isclose(lambda2, eigenvalues[1])}")
```

Рисунок 10: Визуализация собственных значений

```
ВЫЧИСЛЕНИЕ СОБСТВЕННЫХ ЗНАЧЕНИЙ МАТРИЦЫ
================================================================
Матрица A:
[[4 2]
 [1 3]]

Собственные значения: [5. 2.]

Ручное вычисление:
λ₁ = 5.000000
λ₂ = 2.000000
Результаты совпадают: True
```

Рисунок 11: Результат вычисления: собственные значения

```python
# Gráfico
plt.figure(figsize=(6,4))
plt.scatter(np.real(eigenvalues), np.imag(eigenvalues), s=200, c='red')
plt.axhline(y=0, color='k')
plt.axvline(x=0, color='k')
plt.grid()
plt.xlabel('Re(λ)')
plt.ylabel('Im(λ)')
plt.title('Собственные значения матрицы A')
plt.show()
```

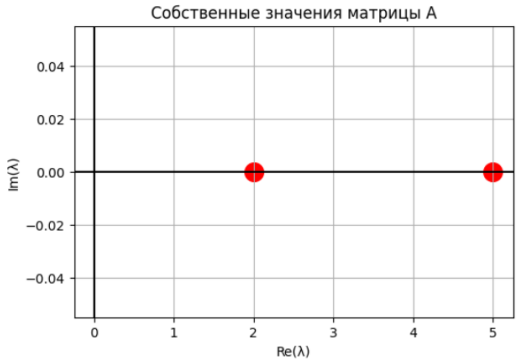

Рисунок 12: Визуализация собственных значений матрицы на комплексной плоск ости

Работа с файлами

```python
import os

# Пункты 1-2: Создание и запись
with open("teste_variante17.txt", "w") as f:
    f.write("Вариант 17: Собственные значения матриц\n")
    f.write("Матрица A = [[4,2],[1,3]] → [5, 2]\n")

# Пункт 3: Чтение файла
with open("teste_variante17.txt", "r") as f:
    print(f.read())

# Пункт 4: Проверка существования
print("Файл существует:", os.path.exists("teste_variante17.txt"))

# Пункт 5: Удаление файла
os.remove("teste_variante17.txt")
print("Файл удален.")
```

```
Вариант 17: Собственные значения матриц
Матрица A = [[4,2],[1,3]] → [5, 2]

Файл существует: True
Файл удален.
```

Рисунок 13: Визуализация собственных значений матрицы на комплексной плосо сти

Магические команды

```
[10]: print(" МАГИЧЕСКИЕ КОМАНДЫ JUPYTER")
print("="*60)

# %lsmagic - список всех магических команд
print("\n1 Список доступных магических команд:")
%lsmagic

# %time - измерение времени
print("\n2 Измерение времени выполнения (%time):")
%time resultado = sum([i**2 for i in range(1000000)])

# %timeit - точное измерение
print("\n3 Точное измерение (%timeit):")
%timeit np.linalg.eigvals(np.random.rand(10,10))

print("\n4 Измерение времени для всей ячейки (%%time):")
```

```
 МАГИЧЕСКИЕ КОМАНДЫ JUPYTER
============================================================

1 Список доступных магических команд:

2 Измерение времени выполнения (%time):
CPU times: total: 391 ms
Wall time: 427 ms

3 Точное измерение (%timeit):
139 µs ± 5.08 µs per loop (mean ± std. dev. of 7 runs, 10,000 loops each)

4 Измерение времени для всей ячейки (%%time):
```

Рисунок 14: магические команды Jupyter

Взаимодействие с оболочкой

```
[8]: %%time
# Создадим большую матрицу
matriz_grande = np.random.rand(50, 50)
autovalores_grandes = np.linalg.eigvals(matriz_grande)
print(f"Найдено {len(autovalores_grandes)} собственных значений")
```

```
Найдено 50 собственных значений
CPU times: total: 0 ns
Wall time: 4.24 ms
```

```
[9]: # %who - список переменных
print("\n5 Текущие переменные в памяти (%who):")
%who

# %env - переменные окружения (сокращенно)
print("\n6 Переменные окружения (первые 5):")
import os
dict(list(os.environ.items())[:5])

print("\n Все магические команды выполнены успешно!")
```

```
5 Текущие переменные в памяти (%who):
A1        a        autovalores      autovalores_grandes      axes     b        c        d        det
eigenvalues        fig      i        idx      imag     lambda1          lambda2          matriz   matriz_grande
matrizes           nome     np       plt      real     resultado        time     trace    verificacao
x         y

6 Переменные окружения (первые 5):

 Все магические команды выполнены успешно!
```

Рисунок 15: магические команды Jupyter

Работа с Google Drive в Google Colab

```python
# Célula 1: Montar o Drive (executa primeiro)
from google.colab import drive
drive.mount('/content/drive')
```

Mostrar saída oculta

```python
# Guardar arquivos LOCALMENTE no Colab
with open("my_text_file.txt", "w") as f:
    f.write("First line\nSecond line\nVariant 17: Matrix eigenvalues\n")

# Criar CSV com nomes
students = [
    ["Pascoal Mendes", 20, "IBT-b-o-24-1"],
    ["Lemos Arnaldo", 22, "IBT-b-o-24-1"],
    ["Danil ", 21, "IBT-b-o-24-1"],
    ["Victor ", 23, "IBT-b-o-24-1"]
]

with open("students.csv", "w") as f:
    f.write("Name,Age,Group\n")
    for s in students:
        f.write(f"{s[0]},{s[1]},{s[2]}\n")
```

Рисунок 16: Подключение Google Drive в Google Colab

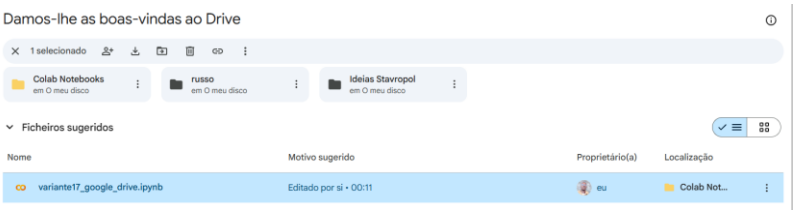

Рисунок 17: Файл создан в Google Диске

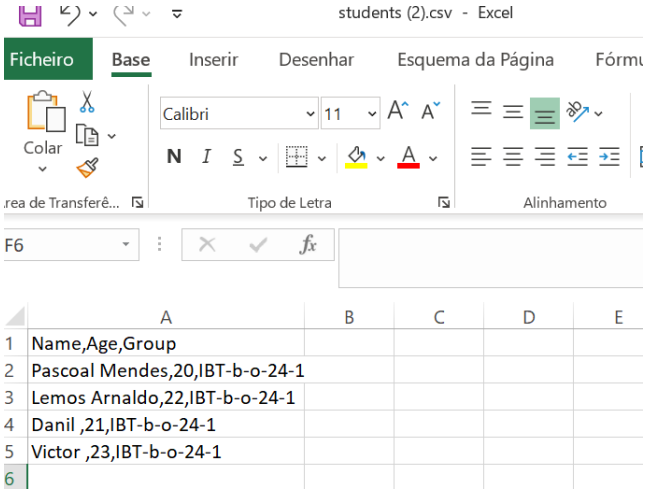

Рисунок 18: список студентов в Excel

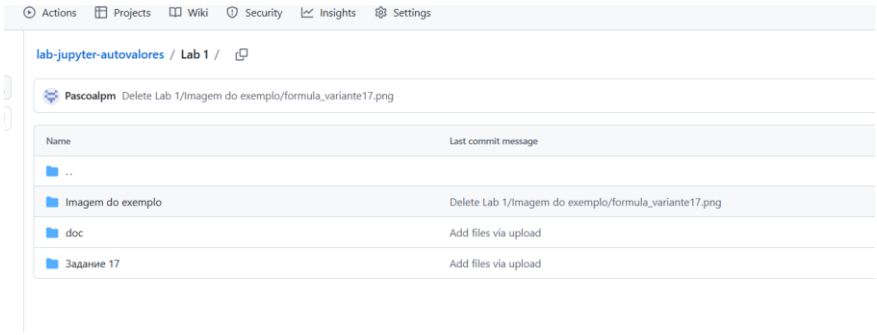

Рисунок 19: Лабораторные работы загружены в репозиторий Github.

Вывод

В ходе лабораторной работы были изучены основные принципы работы с JupyterLab и Google Colab. Полученные знания применены на практике при решении индивидуального задания (вариант 17) по вычислению собственных значений матриц.

1. Какие основные отличия JupyterLab от Jupyter Notebook?

JupyterLab является более продвинутой версией Jupyter Notebook. Основные отличия:

Интерфейс: Jupyter Notebook имеет линейную последовательность ячеек на одной веб-странице, а JupyterLab представляет собой полноценную интегрированную среду (IDE) с вкладками, панелями и файловым менеджером.

Работа с файлами: В Jupyter Notebook можно работать только с .ipynb-файлами, а в JupyterLab поддерживаются разные типы файлов: .ipynb, .py, .csv, .md, .json, .yaml, .txt.

Многозадачность: В Jupyter Notebook нужно открывать несколько вкладок браузера, а в JupyterLab можно работать с

несколькими тетрадями в одном окне благодаря системе вкладок и разделению экрана.

Терминал: JupyterLab имеет встроенный терминал для выполнения команд оболочки, в Jupyter Notebook его нет.

Расширения: JupyterLab имеет встроенный менеджер расширений, что делает добавление новых функций более удобным.

Гибкость интерфейса: В JupyterLab можно изменять расположение панелей и настраивать вид.

2. Как создать новую рабочую среду (ноутбук) в JupyterLab?

Чтобы создать новый ноутбук в JupyterLab:

1. Нажмите кнопку "+" в левом верхнем углу или выберите File → New → Notebook.
2. В открывшемся лаунчере (Launcher) выберите ядро Python (например, "Python 3 (ipykernel)").
3. Новая тетрадь откроется с одной пустой ячейкой.
4. Сразу сохраните её: File → Save Notebook As... и укажите имя файла (расширение .ipynb добавится автоматически).

3. Какие типы ячеек поддерживаются в JupyterLab и как их переключать?

В JupyterLab поддерживаются три основных типа ячеек:

Тип ячейки	Назначение	Горячая клавиша
Code	Для написания и выполнения программного кода	Y
Markdown	Для оформления пояснений, форматированного текста и математических формул	M
Raw	Для хранения необработанного текста (например, для экспорта в другие форматы)	R

Переключение между типами осуществляется двумя способами:

- Через выпадающее меню на панели инструментов

- С помощью горячих клавиш: Y (Code), M (Markdown), R (Raw)

4. Как выполнить код в ячейке и какие горячие клавиши для этого используются?

Для выполнения кода в ячейке используются следующие комбинации клавиш:

Комбинация	Действие
Shift + Enter	Выполнить ячейку и перейти к следующей (создать новую, если текущая последняя)
Ctrl + Enter	Выполнить ячейку и остаться на ней
Alt + Enter	Выполнить ячейку и вставить новую ниже

Также можно выполнить код через меню Cell → Run Cells или нажав на кнопку на панели инструментов.

5. Как запустить терминал или текстовый редактор внутри JupyterLab?

Для запуска терминала:

- File → New → Terminal
- Или в лаунчере (Launcher) нажмите на иконку Terminal

Для запуска текстового редактора:

- File → New → Text File (для нового файла)
- Или дважды кликните на любом текстовом файле (.py, .txt, .md) в файловом менеджере слева

Терминал позволяет выполнять команды оболочки прямо в интерфейсе JupyterLab, а текстовый редактор — редактировать файлы различных типов.

6. Какие инструменты JupyterLab позволяют работать с файлами и структурами каталогов?

JupyterLab предоставляет следующие инструменты для работы с файлами:

- Файловый менеджер (File Browser) – панель слева, показывающая структуру папок и файлов. Позволяет:
    - Создавать, переименовывать, удалять файлы и папки
    - Загружать файлы с компьютера
    - Копировать и перемещать файлы
    - Просматривать содержимое файлов
- Терминал – для выполнения команд оболочки (создание папок, копирование, удаление, перемещение файлов)
- Редактор – для просмотра и редактирования файлов различных типов (текстовые файлы, Python-скрипты, JSON, Markdown и др.)

7. Как можно управлять ядрами (kernels) в JupyterLab?

Управление ядрами осуществляется через меню Kernel:

Действие	Путь в меню	Горячая клавиша
Перезапустить ядро	Kernel → Restart Kernel	0 + 0
Прервать выполнение	Kernel → Interrupt	I + I
Перезапустить и выполнить все	Kernel → Restart Kernel and Run All Cells	—
Переключить ядро	Kernel → Change Kernel	—
Выключить ядро	Kernel → Shutdown Kernel	—

Индикатор ядра находится в правой части панели меню:

- Полый кружок — ядро в режиме ожидания
- Закрашенный кружок — ядро выполняет задачу

8. **Каковы основные возможности системы вкладок и окон в интерфейсе JupyterLab?**

JupyterLab поддерживает гибкую систему вкладок и окон:

- Вкладки: Можно открыть несколько тетрадей, текстовых файлов или терминалов в разных вкладках одной панели.

- Разделение экрана: Можно перетаскивать вкладки для создания нескольких панелей рядом (сплит-режим).
- Перетаскивание: Вкладки можно перемещать между панелями или создавать новые окна.
- Закрепление: Некоторые панели (например, файловый менеджер) можно закрепить слева или справа.
- Настройка размеров: Границы между панелями можно перетаскивать для изменения размеров.
- Контекстное меню: Правый клик на вкладке открывает дополнительные опции.

9. Какие магические команды можно использовать в JupyterLab для измерения времени выполнения кода? Приведите примеры.

Для измерения времени выполнения кода используются следующие магические команды:

Команда	Описание	Пример
%time	Измеряет время выполнения одной строки	%time sum(range(1000000))
%timeit	Запускает код много раз (по умолчанию 100000) и выводит среднее время трёх наиболее быстрых прогонов	%timeit np.linalg.eigvals(np.random.rand(10,10))
%%time	Измеряет время выполнения всей ячейки (должна быть первой строкой ячейки)	%%time

10. Какие магические команды позволяют запускать код на других языках программирования в JupyterLab?

JupyterLab поддерживает магические команды для запуска кода на других языках:

Команда	Язык
%%bash	Bash/shell
%%perl	Perl

%%ruby	Ruby
%%script	Общая команда для скриптов (можно указать интерпретатор)
%%html	HTML
%%javascript или %%js	JavaScript
%%latex	LaTeX
%%python2	Python 2
%%python3	Python 3

```
%%bash
echo "Привет из Bash!"
ls -la
```

Эти команды позволяют использовать JupyterLab как мультиязычную среду разработки.

11. Какие основные отличия Google Colab от JupyterLab?

Основные отличия Google Colab от JupyterLab:

Характеристика	Google Colab	JupyterLab
Среда	Облачная (требуется интернет)	Локальная (на вашем компьютере)
Оборудование	Бесплатный доступ к GPU и TPU	Только CPU (если нет локального GPU)
Установка	Не требуется, работает в браузере	Требуется установка Python и Jupyter
Интеграция	С Google Диском	С локальной файловой системой
Совместная работа	Поддерживается (как Google Docs)	Ограничена
Стоимость	Бесплатно (с ограничениями по времени)	Бесплатно
Сессия	Автоматически завершается через 90 минут простоя	Работает, пока запущен сервер
Расширения	Ограниченная поддержка	Полная поддержка расширений

12. Как создать новый ноутбук в Google Colab?

Чтобы создать новый ноутбук в Google Colab:

1. Перейдите на сайт colab.research.google.com
2. В появившемся окне выберите "Новый ноутбук" (New Notebook)
3. Или через меню: Файл → Новый ноутбук

Также можно:

- Загрузить существующий .ipynb файл с компьютера
- Открыть ноутбук с GitHub (через Файл → Открыть ноутбук → GitHub)
- Создать копию существующего ноутбука с Google Диска Ноутбук автоматически сохраняется на Google Диске в папку "Colab Notebooks".

13. Какие типы ячеек доступны в Google Colab, и как их переключать?

В Google Colab доступны следующие типы ячеек:

Тип ячейки	Назначение	Горячая клавиша
Код (Code)	Для написания и выполнения Python-кода	Ctrl + M Y
Текст (Markdown)	Для оформления документации, пояснений и формул LaTeX	Ctrl + M M

Переключение типов осуществляется:

- Через выпадающее меню в панели инструментов ячейки
- С помощью горячих клавиш (указаны выше)

В Google Colab также поддерживается форматирование текста с помощью Markdown, включая заголовки, списки, жирный и курсивный текст, математические формулы.

14. Как выполнить код в ячейке Google Colab и какие горячие клавиши для этого используются?

В Google Colab используются следующие комбинации клавиш для выполнения кода:

Комбинация	Действие
Shift + Enter	Выполнить ячейку и перейти к следующей
Ctrl + Enter	Выполнить ячейку и остаться на ней
Alt + Enter	Выполнить ячейку и вставить новую ниже

Также можно выполнить код:

- Через меню Среда выполнения → Выполнить ячейку
- Нажав на значок слева от ячейки
- Через меню Среда выполнения → Выполнить все для выполнения всех ячеек

15. Какие способы загрузки и сохранения файлов поддерживает Google Colab?

Google Colab поддерживает несколько способов работы с файлами:

Загрузка файлов:

- Через боковую панель: Файлы → Загрузить файлы (иконка)
- С помощью Python: from google.colab import files; files.upload()
- С Google Диска (после монтирования)
- С GitHub (через меню Файл → Открыть ноутбук → GitHub)

Сохранение файлов:

- На Google Диск (после монтирования): drive.mount('/content/drive')
- Скачивание на компьютер: files.download('filename')
- Сохранение копии на Google Диске: Файл → Сохранить копию на Google Диске
- Экспорт ноутбука: Файл → Скачать как → .ipynb

16. Как можно подключить Google Drive к Google Colab и работать с файлами?

Подключение Google Drive к Colab выполняется в несколько шагов:

```
from google.colab import drive
drive.mount('/content/drive')
```

После выполнения появится ссылка для авторизации. Нужно:

1. Перейти по ссылке
2. Выбрать аккаунт Google
3. Скопировать код авторизации
4. Вставить код в Colab и нажать Enter

После успешного подключения:

- Файлы Google Диска доступны по пути /content/drive/MyDrive/
- Можно читать и записывать файлы как с обычной файловой системой:

```
# Чтение файла
with open('/content/drive/MyDrive/файл.txt', 'r') as f:
    content = f.read()
# Запись файла
with open('/content/drive/MyDrive/новый_файл.txt', 'w') as f:
    f.write('Текст')
```

17. Какие команды используются для загрузки файлов в Google Colab из локального компьютера?

Для загрузки файлов с локального компьютера используется специальный модуль google.colab.files:

```
from google.colab import files
uploaded = files.upload()
```

После выполнения этой команды:

1. Откроется диалоговое окно для выбора файлов
2. Можно выбрать один или несколько файлов
3. Файлы загружаются в текущую директорию Colab
4. Результат сохраняется в словаре uploaded (ключ — имя файла, значение — содержимое)

Пример обработки загруженного файла:

```
for filename, content in uploaded.items():
    print(f'Загружен файл: {filename} ({len(content)} байт)')
    # Сохраняем на диск
    with open(filename, 'wb') as f:
        f.write(content)
```

18. Как посмотреть список файлов, хранящихся в среде Google Colab?

В Google Colab можно посмотреть список файлов несколькими способами:

1. Через боковую панель (графический интерфейс):

- Нажмите на значок (Файлы) в левой части экрана
- Откроется файловый менеджер со всеми файлами и папками

2. С помощью команд терминала (префикс !):

```
!ls -la        # показать все файлы (включая скрытые)
!ls -lh        # показать с размерами файлов в человеко-читаемом формате
!pwd           # показать текущую директорию
!ls /content   # показать содержимое папки /content
```

3. С помощью Python:

```
import os
print("Файлы в текущей директории:", os.listdir())
print("Файлы на Google Диске:", os.listdir('/content/drive/MyDrive'))
```

19. Какие магические команды можно использовать в Google Colab для измерения времени выполнения кода? Приведите примеры.

В Google Colab поддерживаются те же магические команды для измерения времени, что и в Jupyter:

Команда	Описание
%time	Время выполнения одной строки
%timeit	Среднее время многократного выполнения
%%time	Время выполнения всей ячейки (первая строка)

Примеры из лабораторной работы:

```
# Пример %time
%time resultado = sum([i**2 for i in range(1000000)])
# Вывод: CPU times: user 391 ms, sys: 0 ns, total: 391 ms, Wall time : 427 ms

# Пример %timeit
%timeit np.linalg.eigvals(np.random.rand(10,10))
# Вывод: 139 µs ± 5.08 µs per loop (mean ± std. dev. of 7 runs, 10,000 loops each)

# Пример %%time
%%time
matriz_grande = np.random.rand(50, 50)
autovalores = np.linalg.eigvals(matriz_grande)
print(f"Найдено {len(autovalores)} собственных значений")
# Вывод: CPU times: user 48 ms, sys: 0 ns, total: 48 ms, Wall time: 52.3 ms
```

20. Как можно изменить аппаратные ресурсы в Google Colab (например, переключиться на GPU)?

Чтобы изменить аппаратные ресурсы в Google Colab:

1. Перейдите в меню: Среда выполнения → Изменить среду выполнения
2. В открывшемся окне в разделе "Аппаратный ускоритель" (Hardware accelerator) выберите:

- GPU (для графического ускорения, обычно NVIDIA Tesla T4 или K80)
- TPU (для ускорения TensorFlow)
- Нет (CPU, используется по умолчанию)

3. Нажмите кнопку "Сохранить"

Проверка наличия GPU:

```
import torch
print("GPU доступен:", torch.cuda.is_available())
if torch.cuda.is_available():
    print("Имя GPU:", torch.cuda.get_device_name(0))
```

Проверка наличия TPU:

```
import os
print("TPU доступен:", 'COLAB_TPU_ADDR' in os.environ)
```